

6.4 树和森林

6.4.1 树和森林的存储方式

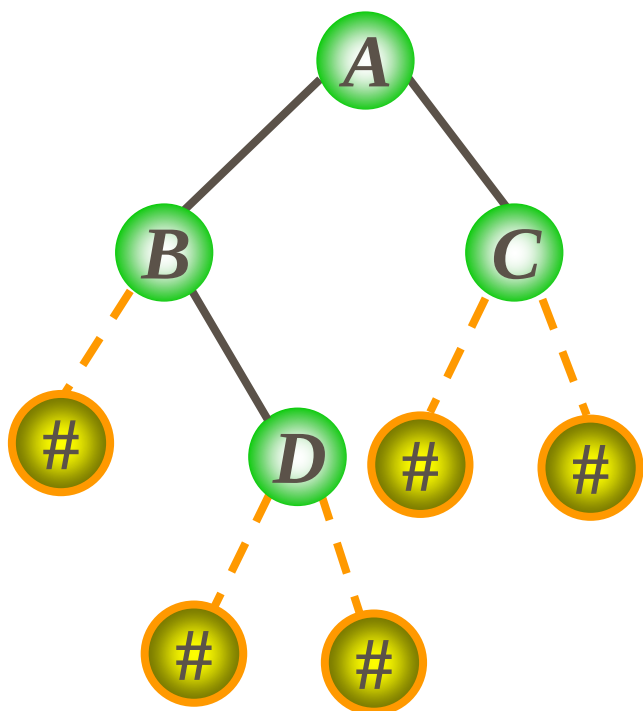
6.4.2 树和森林与二叉树的转换

6.4.3 树和森林的遍历

回顾

遍历的性质

- 性质1、由先序序列和**中序**序列可惟一确定这棵二叉树
- 性质2、由后序序列和**中序**序列可惟一确定这棵二叉树



扩展二叉树的先序遍历序列

A B # D # # C # #



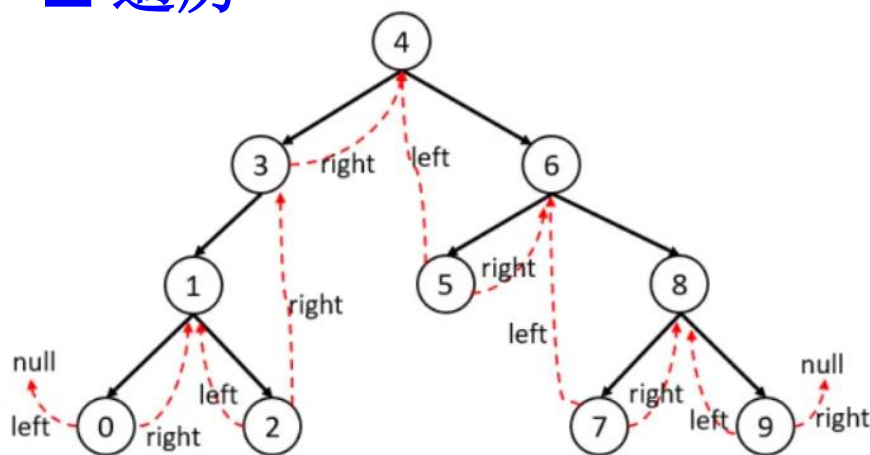
构造二叉树

回顾

线索二叉树

- 先序线索二叉树
- 中序线索二叉树
- 后序线索二叉树
- 遍历

每次只修改前驱结点的右指针（后继）和本结点的左指针（前驱）



中序遍历结果：[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

第 1 步：从根节点 4 开始，一直往左走，找到节点 0。

第 2 步：打印节点 0 的值，节点 0 没有右子节点，到他的后继节点 1。

第 3 步：打印节点 1 的值，节点 1 有右子节点，到他的右子节点 2。
一直往左走，找到最左边节点 2（也就是他自己）。

第 4 步：打印节点 2 的值，节点 2 没有右子节点，到他的后继节点 3。

第 5 步：打印节点 3 的值，节点 3 没有右子节点，到他的后继节点 4。

第 6 步：打印节点 4 的值，节点 4 有右子节点，到他的右子节点 6。
一直往左走，找到最左边节点 5。

第 7 步：打印节点 5 的值，节点 5 没有右子节点，到他的后继节点 6。

第 8 步：打印节点 6 的值，节点 6 有右子节点，到他的右子节点 8。
一直往左走，找到最左边节点 7。

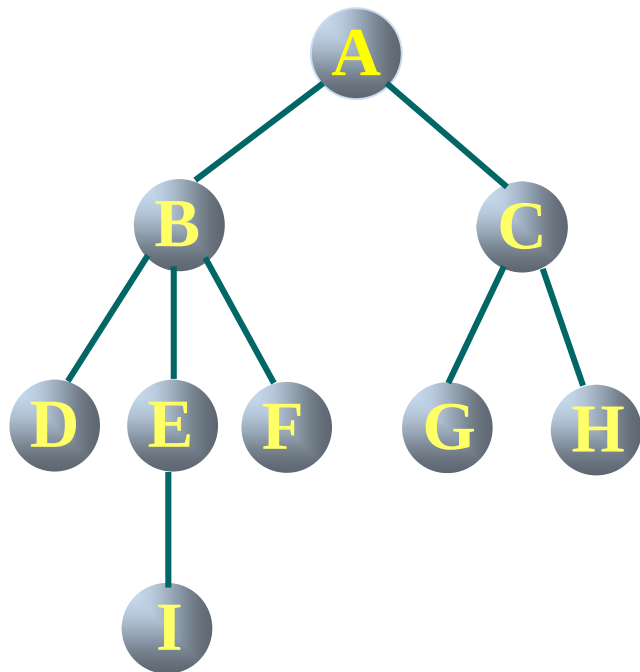
第 9 步：打印节点 7 的值，节点 7 没有右子节点，到他的后继节点 8。

第 10 步：打印节点 8 的值，节点 8 有右子节点，到他的右子节点 9。
一直往左走，找到最左边节点 9（也就是他自己）。

第 11 步：打印节点 9 的值，节点 9 没有右子节点，他的后继节点为空，
循环结束。

回顾

1、双亲表示法



找某一结点的双亲，按照该结点的
双亲下表即可找到。但求某一
结点的孩子，要遍历整个结构。

0	A	-1
1	B	0
2	C	0
3	D	1
4	E	1
5	F	1
6	G	2
7	H	2
8	I	4

回顾

2、孩子表示法

方案一：指针域的个数等于树的度

data	child1	child2		childd
------	--------	--------	-------	--------

方案二：指针域的个数等于该结点的度

data	degree	child1	child2		childd
------	--------	--------	--------	-------	--------

方案三：将结点的所有孩子放在一起，构成线性表。

孩子结点：

child	next
-------	------

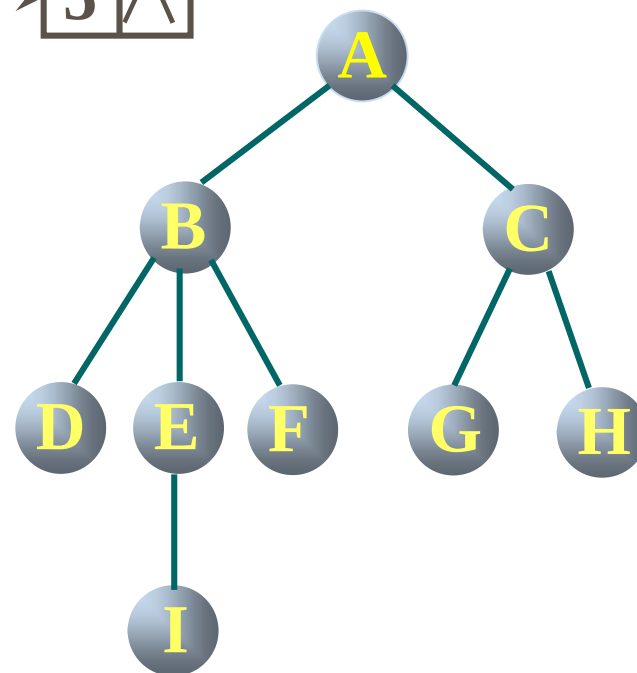
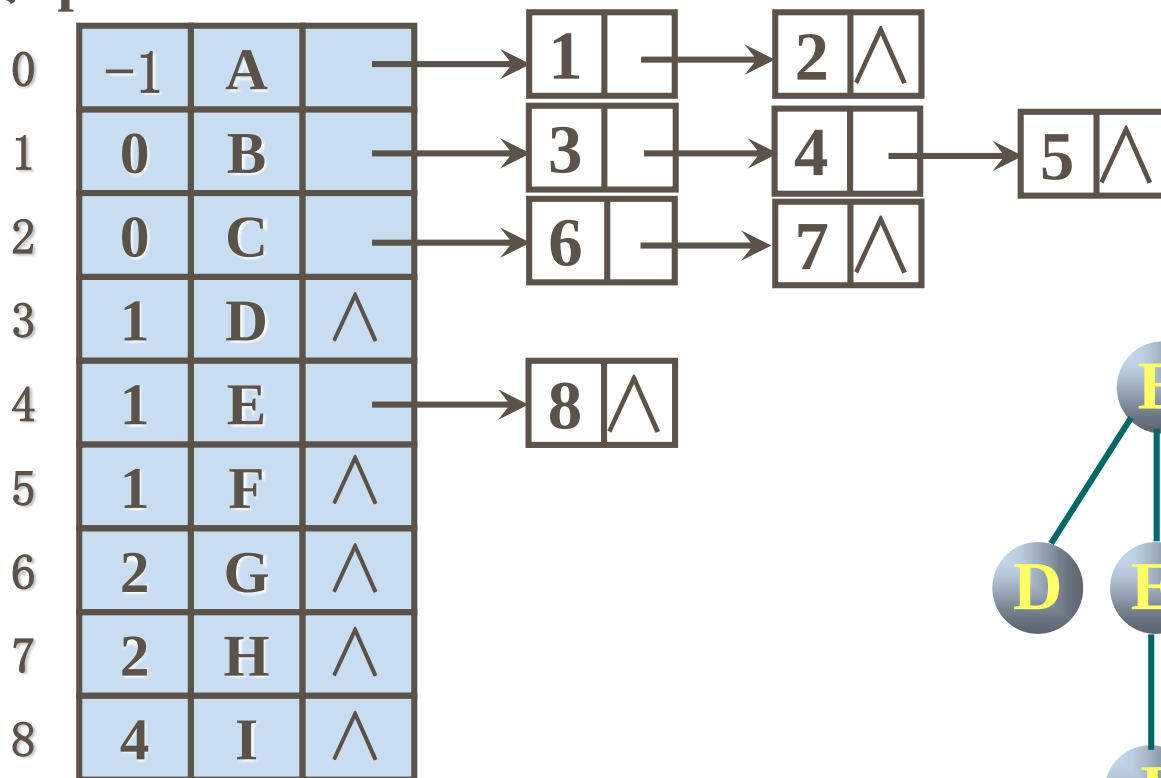
表头结点：

data	firstchild
------	------------

回顾

带双亲的孩子链表：将双亲表示法和孩子表示法结合

下标 para data firstchild



6.4.1 树和森林的存储方式

3、孩子兄弟表示法—又称二叉树表示法

结点结构

firstchild	data	nextsibling
------------	------	-------------

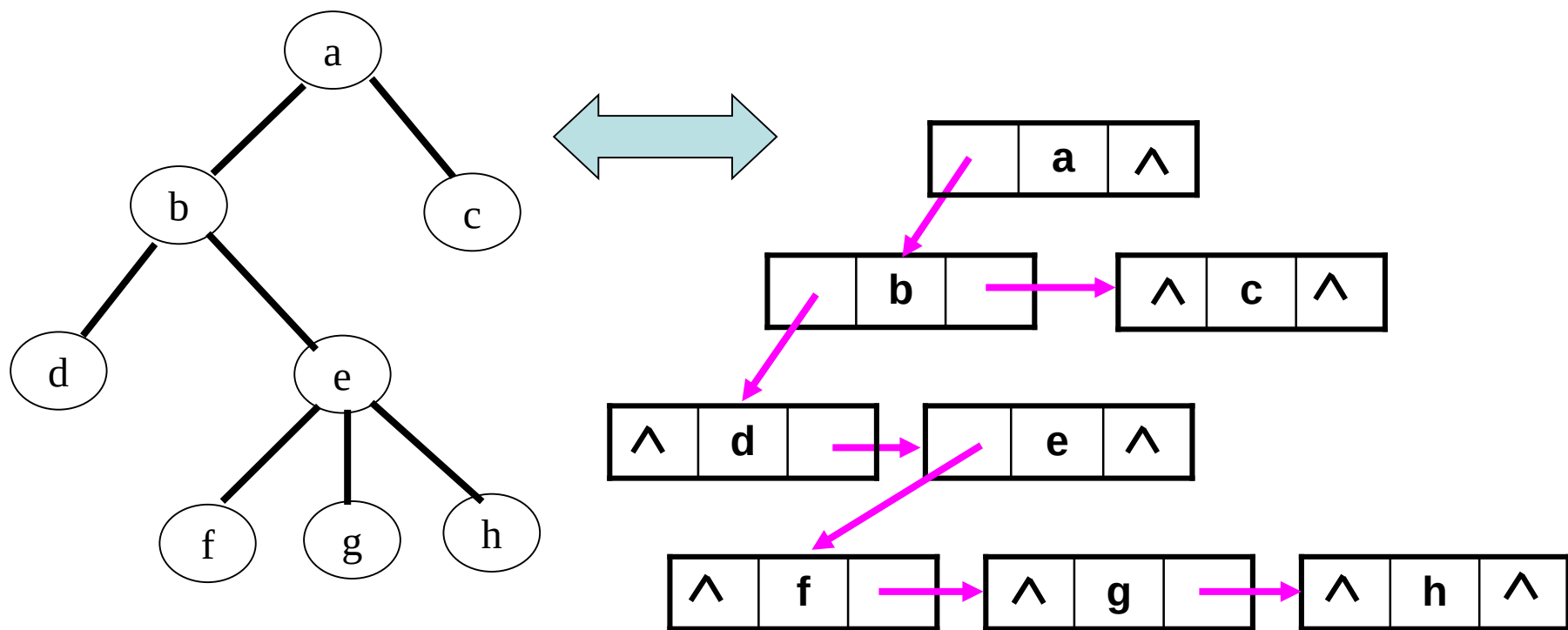
data: 数据域，存储该结点的数据信息；

firstchild: 指针域，指向该结点第一个孩子；

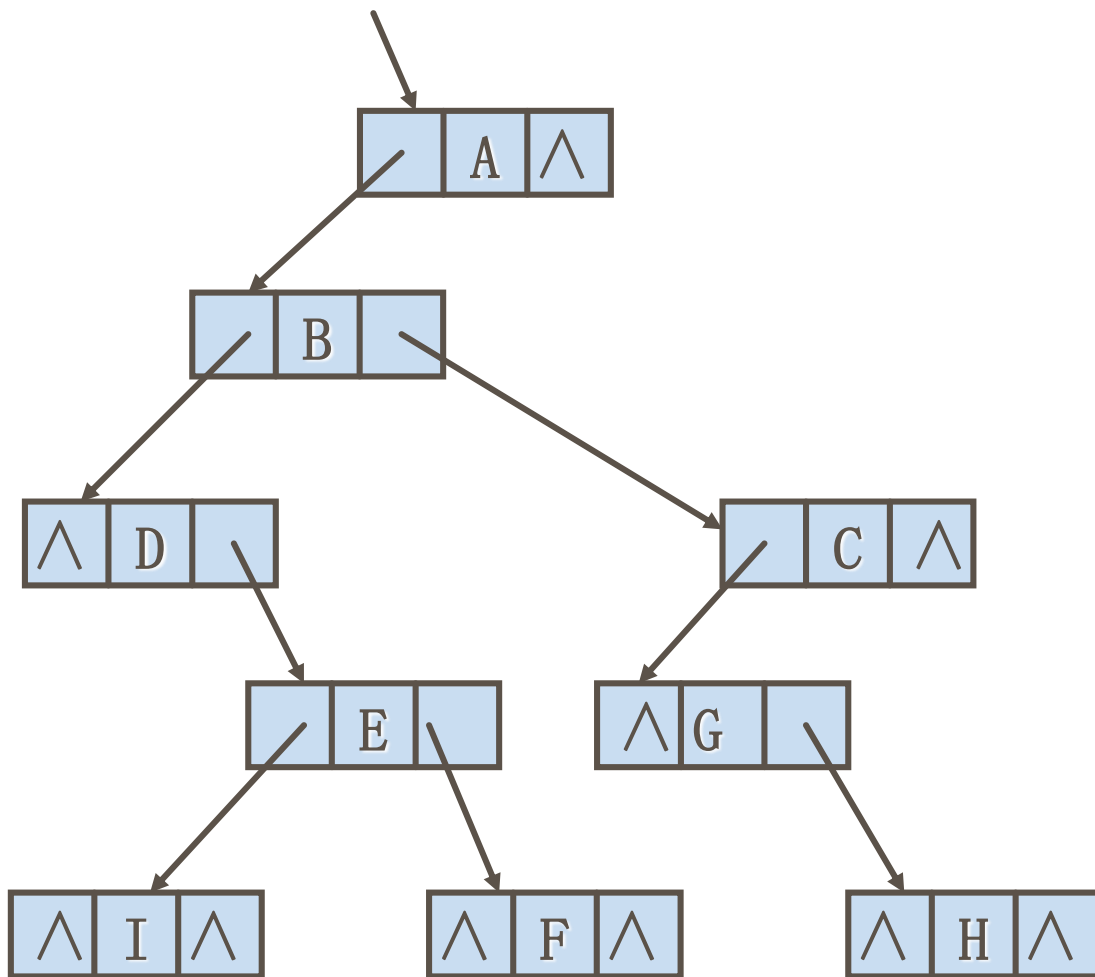
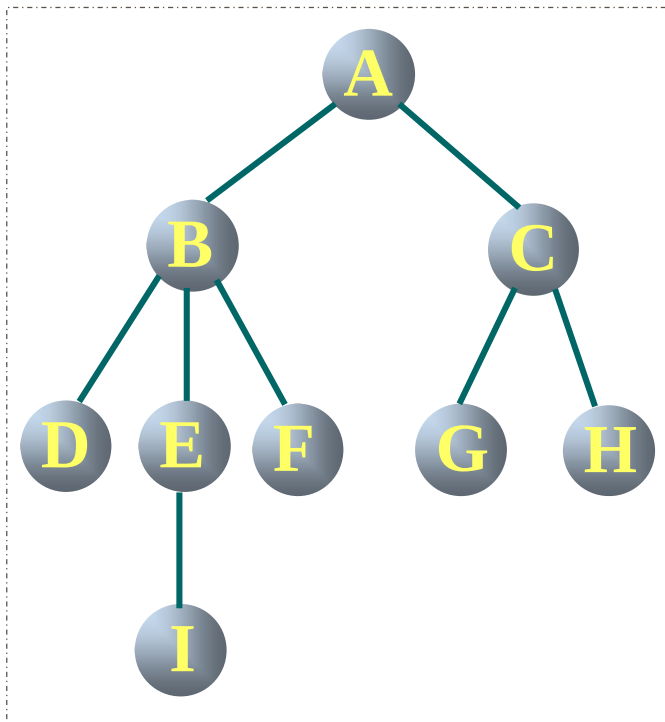
nextsibling: 指针域，指向该结点的右兄弟结点。

```
typedef struct CSNode {  
    ElemType      data;  
    struct CSNode *firstchild, *nextsibling;  
}CSNode,*CSTree;
```

6.4.1 树和森林的存储方式



6.4.1 树和森林的存储方式



6.4 树和森林

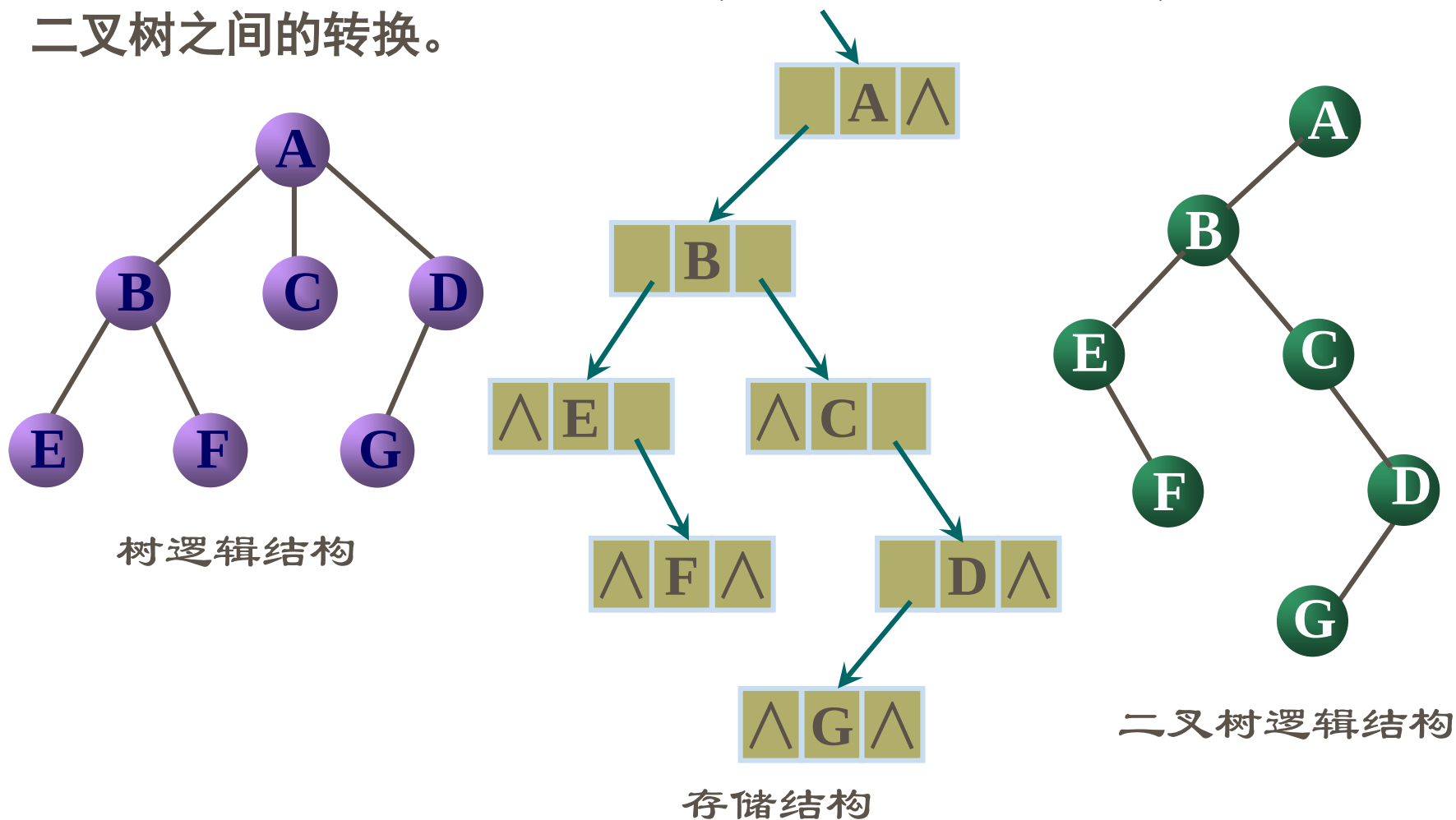
6.4.1 树和森林的存储方式

6.4.2 树和森林与二叉树的转换

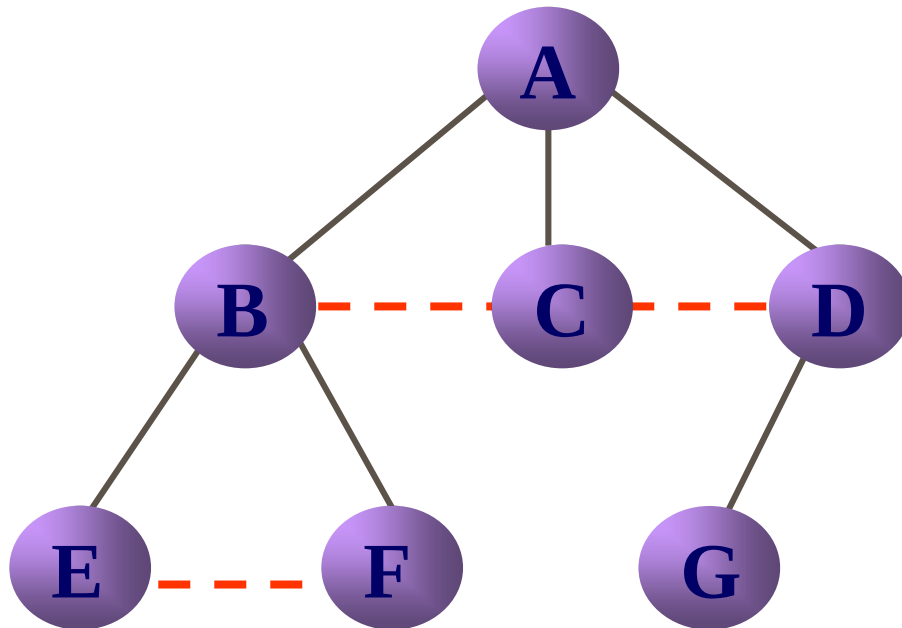
6.4.3 树和森林的遍历

6.4.2 树与二叉树的转换

二叉树与树都可用二叉链表存贮，以二叉链表作中介，可导出树与二叉树之间的转换。

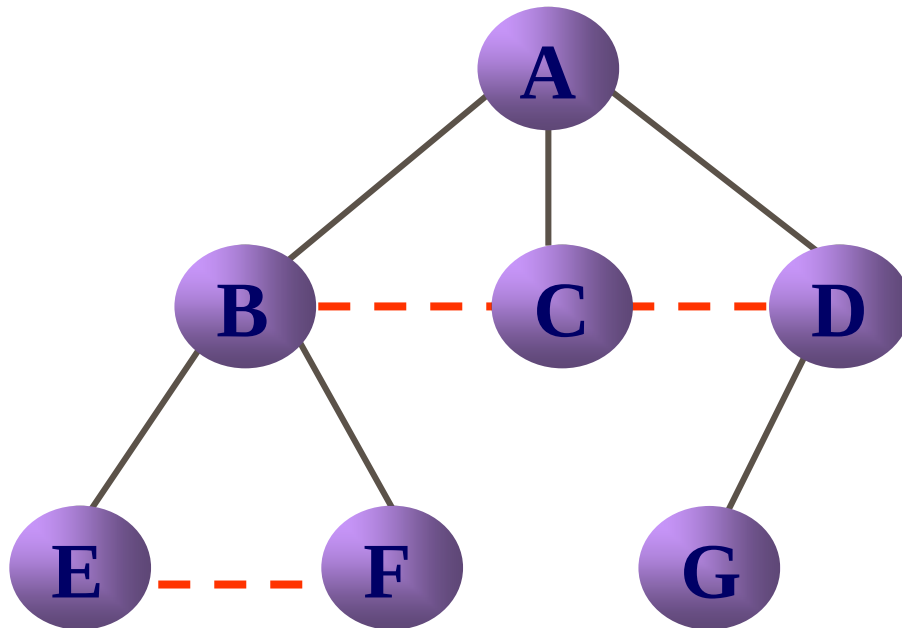


6.4.2 树与二叉树的转换



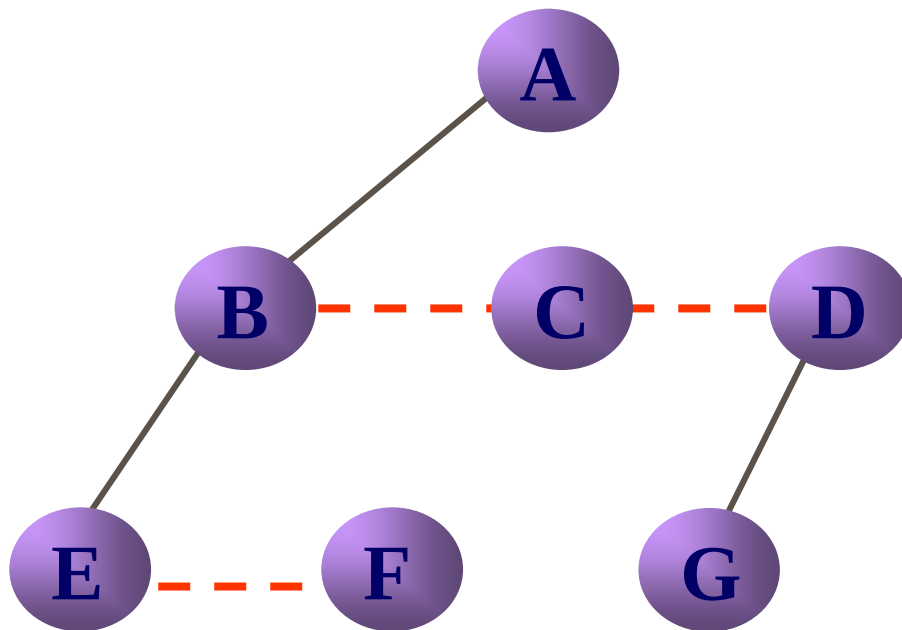
1. 兄弟加线.

6.4.2 树与二叉树的转换



1. 兄弟加线.
2. 保留双亲与第一孩子连线, 删去与其他孩子的连线.

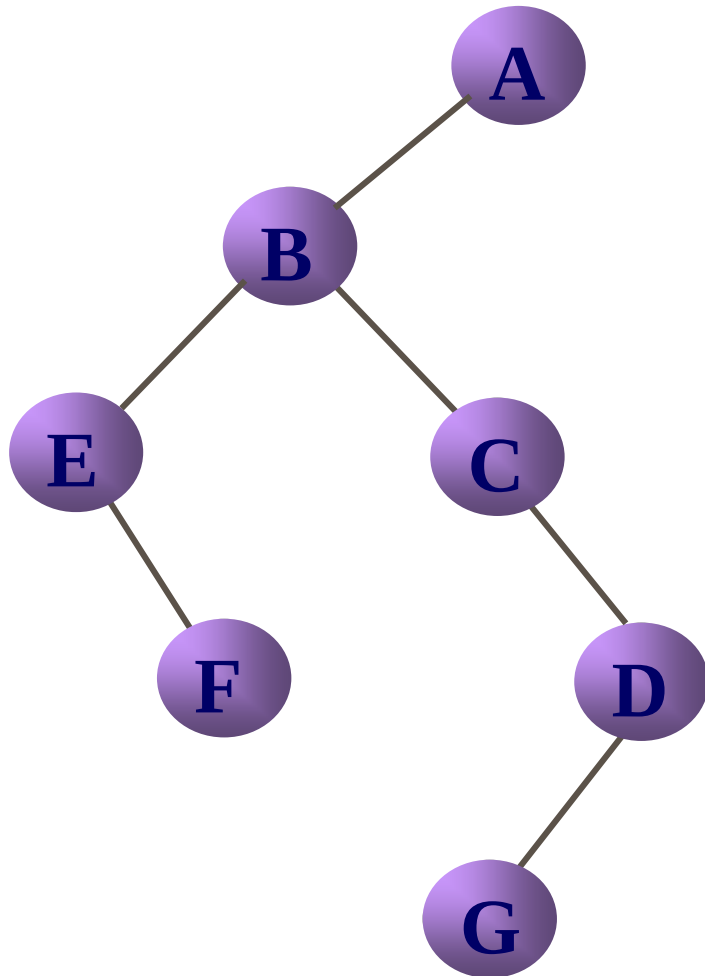
6.4.2 树与二叉树的转换



1. 兄弟加线.
2. 保留双亲与第一孩子连线, 删去与其他孩子的连线.
3. 顺时针旋转45度, 使之层次分明.

以根结点为轴; 左孩子(只有一个孩子)不再旋转

6.4.2 树与二叉树的转换



1. 兄弟加线.
2. 保留双亲与第一孩子连线, 删去与其他孩子的连线.
3. 顺时针旋转 45 度, 使之层次分明.

以根结点为轴; 左孩子(只有一个孩子)不再旋转

6.4.2 树与二叉树的转换

- (1)加线——树中所有相邻兄弟之间加一条连线。
- (2)去线——对树中的每个结点，只保留它与第一个孩子结点之间的连线，删去它与其它孩子结点之间的连线。
- (3)层次调整——以根结点为轴心，将树顺时针转动一定的角度，使之层次分明。

6.4.2 树与二叉树的转换

讨论：森林如何转为二叉树？

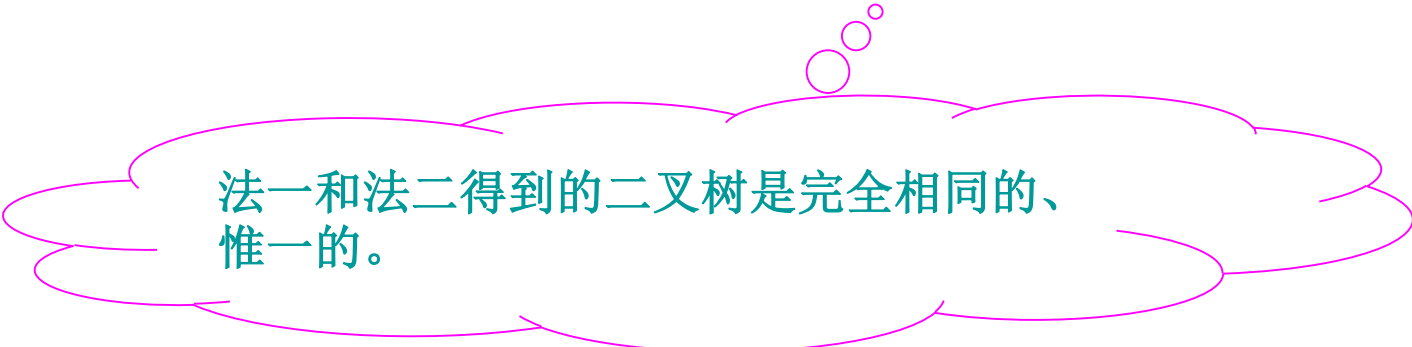
即 $F = \{T_1, T_2, \dots, T_m\} \longleftrightarrow B = \{\text{root}, \text{LB}, \text{RB}\}$

法一：

- ① 各森林先各自转为二叉树；
- ② 依次连到前一个二叉树的右子树上。

法二：森林直接变兄弟，再转为二叉树

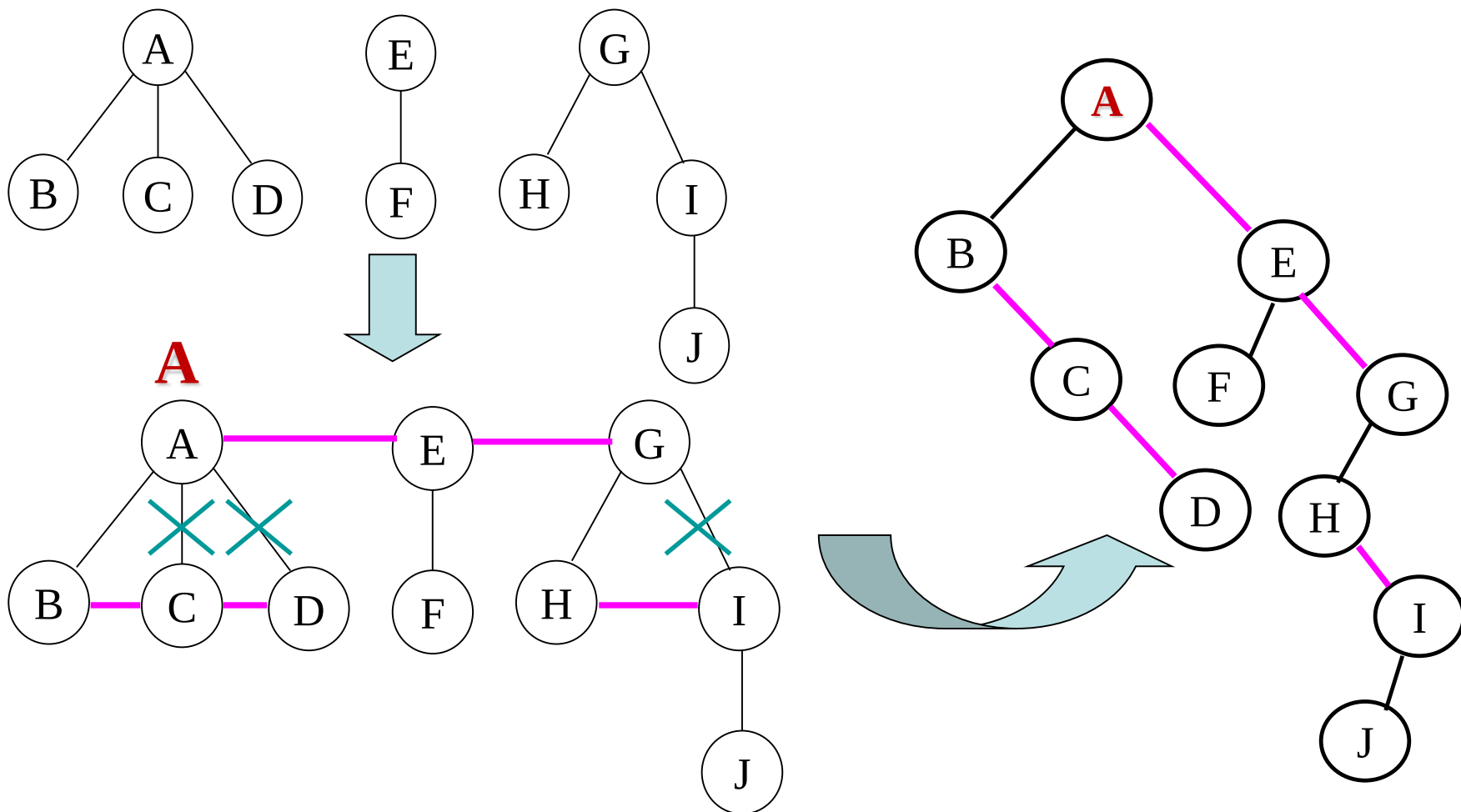
（参见教材P138图6.17，两种方法都有转换示意图）



法一和法二得到的二叉树是完全相同的、惟一的。

6.4.2 树与二叉树的转换

(用法二，森林直接变兄弟，再转为二叉树)



兄弟相连 长兄为父，头树为根 孩子靠左

6.4.2 树与二叉树的转换

(1) 树转化成二叉树的简单方法

- ① 在同胞兄弟之间加连线;
- ② 保留结点与第一个孩子之间的连线, 去掉其余连线;
- ③ 顺时针旋转45度。
 - 以根结点为轴; 左孩子(只有一个孩子)不再旋转。

森林转化成二叉树

- ① 将森林中的每棵树转换成对应的二叉树;
- ② 将森林中已经转换成的二叉树的各个根视为兄弟, 各兄弟之间自第一棵树根到最后一棵树根之间加连线;
- ③ 以第一棵树的根为轴, 顺时针旋转45度。

6.4.2 树与二叉树的转换

(2) 森林转化成二叉树的规则

① 若F为空，即 $n = 0$ ，则

对应的二叉树B为空二叉树。

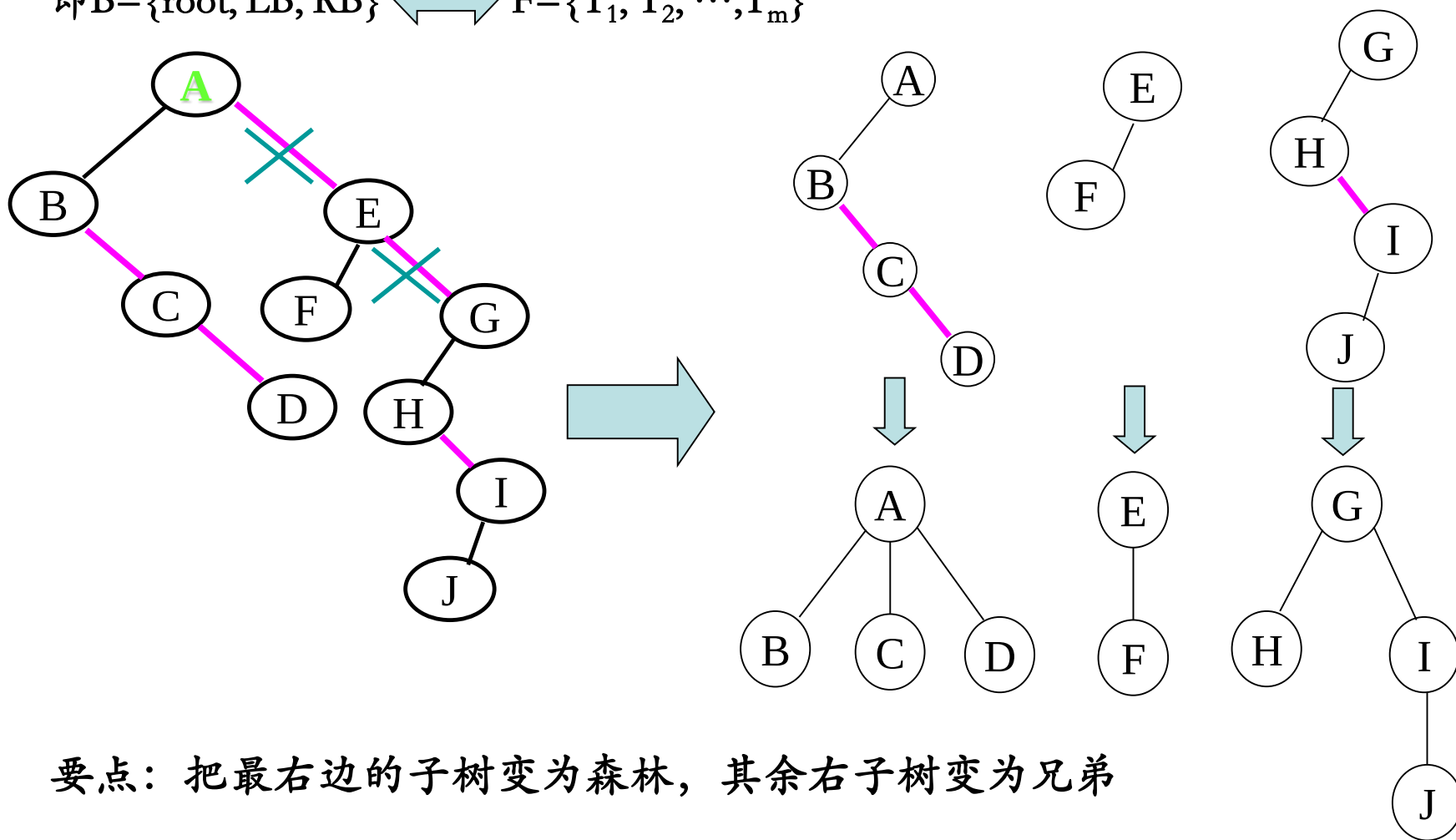
② 若F不空，则

- 对应二叉树B的根 $\text{root}(B)$ 是F中第一棵树 T_1 的根 $\text{root}(T_1)$ ；
- 其左子树为 $B(T_{11}, T_{12}, \dots, T_{1m})$ ，其中， $T_{11}, T_{12}, \dots, T_{1m}$ 是 $\text{root}(T_1)$ 的子树；
- 其右子树为 $B(T_2, T_3, \dots, T_n)$ ，其中， $T_2, T_3, \dots, T_n$ 是除 T_1 外其它树构成的森林。

6.4.2 树与二叉树的转换

讨论：二叉树如何还原为森林？

即 $B = \{\text{root}, \text{LB}, \text{RB}\} \longleftrightarrow F = \{T_1, T_2, \dots, T_m\}$



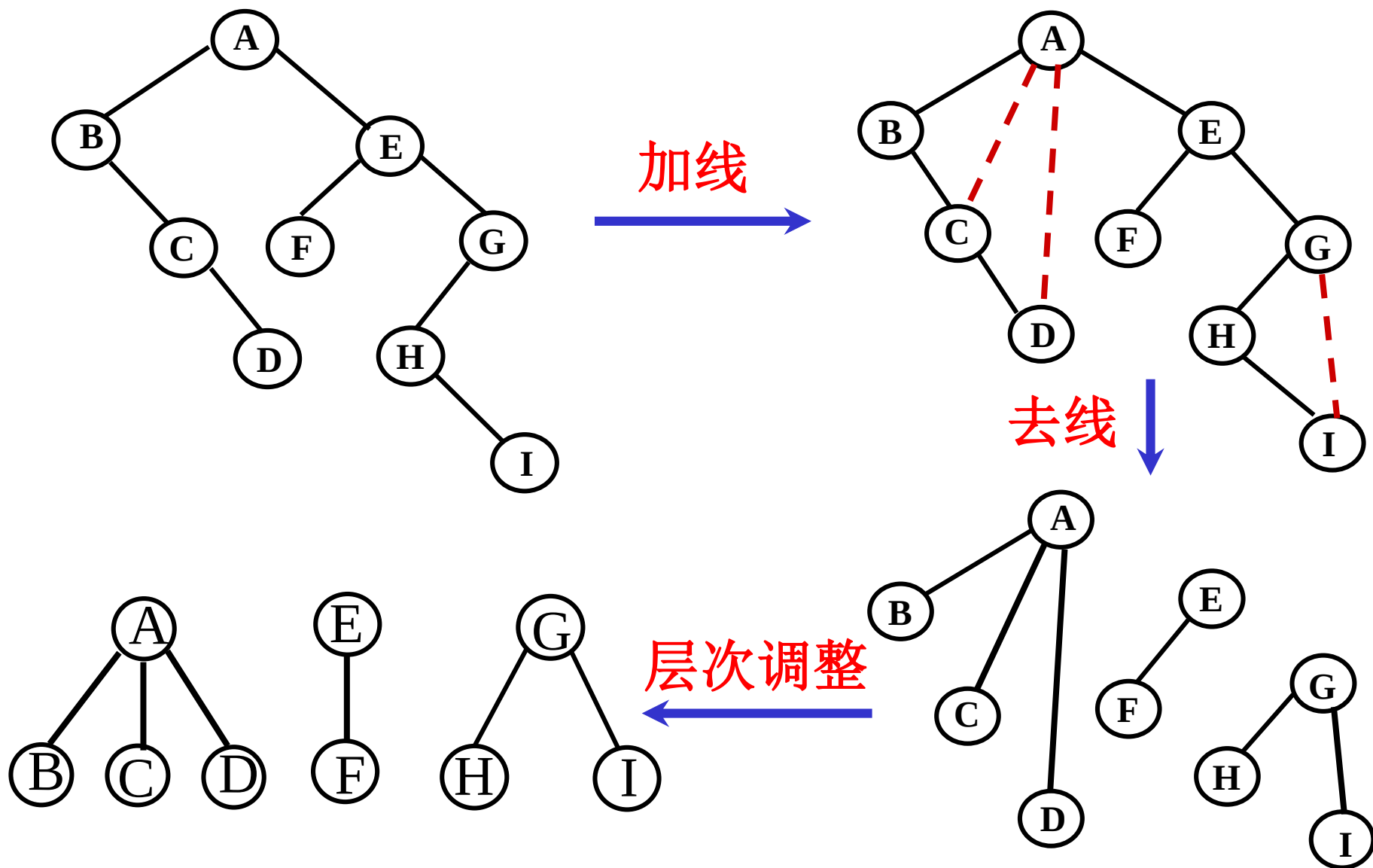
要点：把最右边的子树变为森林，其余右子树变为兄弟

6.4.2 树与二叉树的转换

二叉树转换为树或森林

- (1) 加线——若某结点 x 是其双亲 y 的左孩子，则把结点 x 的右孩子、右孩子的右孩子、……，都与结点 y 用线连起来；
- (2) 去线——删去原二叉树中所有的双亲结点与右孩子结点的连线；
- (3) 层次调整——整理由(1)、(2)两步所得到的树或森林，使之层次分明。

6.4.2 树与二叉树的转换



6.4.2 树与二叉树的转换

(3) 二叉树转换为森林的规则

- ① 如果B为空，则对应的森林F也为空。
- ② 如果B非空，则F中第一棵树 T_1 的根为root； T_1 的根的子树森林 $\{ T_{11}, T_{12}, \dots, T_{1m} \}$ 是由root 的左子树 LB 转换而来，F 中除了 T_1 之外其余的树组成的森林 $\{ T_2, T_3, \dots, T_n \}$ 是由 root的右子树 RB 转换而成的森林。

6.4.2 树与二叉树的转换

树和二叉树之间的对应关系

树：兄弟关系

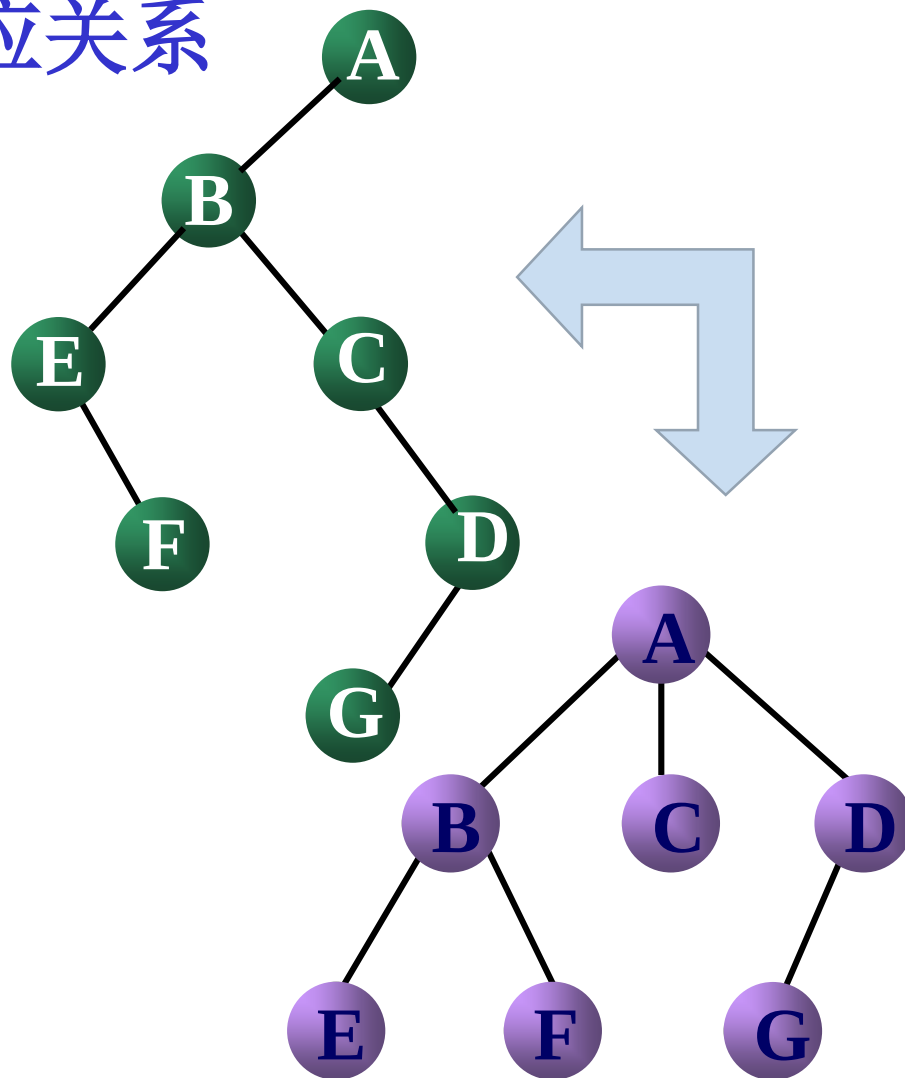


二叉树：双亲和右孩子

树：双亲和长子

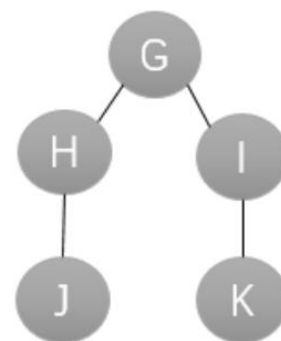
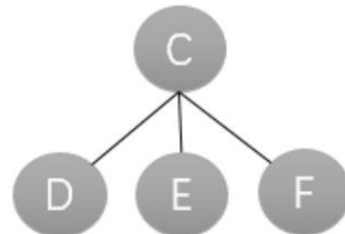
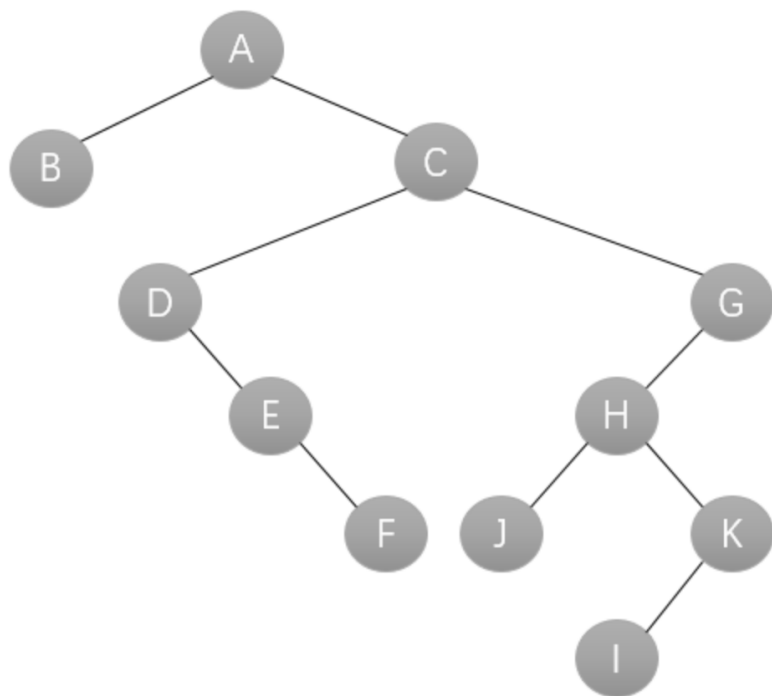


二叉树：双亲和左孩子



6.4.2 树与二叉树的转换

例：把下面二叉树转化为森林



6.4 树和森林

6.4.1 树和森林的存储方式

6.4.2 树和森林与二叉树的转换

6.4.3 树和森林的遍历

6.4.3 树和森林的遍历

树的遍历 { 深度优先遍历 (先根、后根) 树没有中序遍历 (因子树不分左右)
广度优先遍历 (层次)

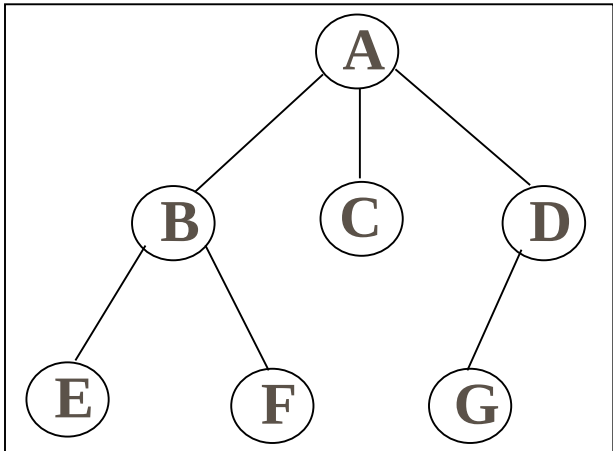
1、先根遍历

- 访问根结点;
- 依次先根遍历根结点的每棵子树。

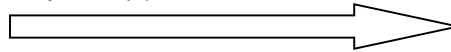
2、后根遍历

- 依次后根遍历根结点的每棵子树;
- 访问根结点。

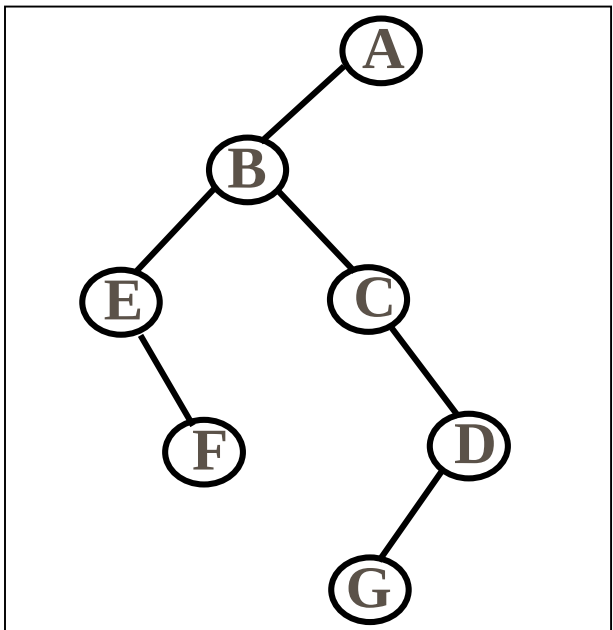
6.4.3 树和森林的遍历



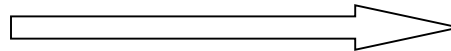
先根遍历



ABEFCDG



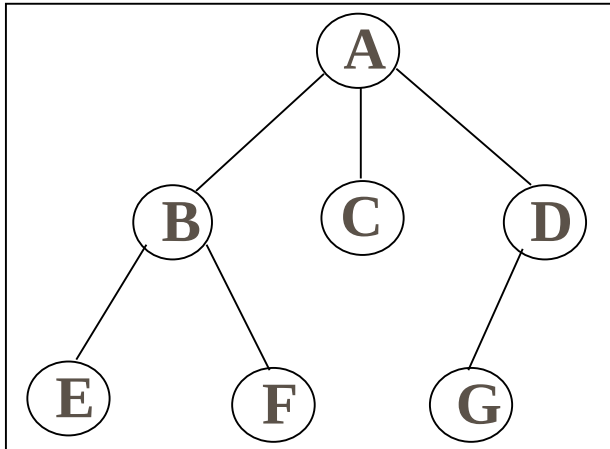
先序遍历



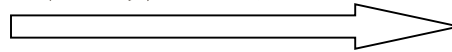
ABEFCDG

树的先根遍历等价于二叉树的先序遍历!

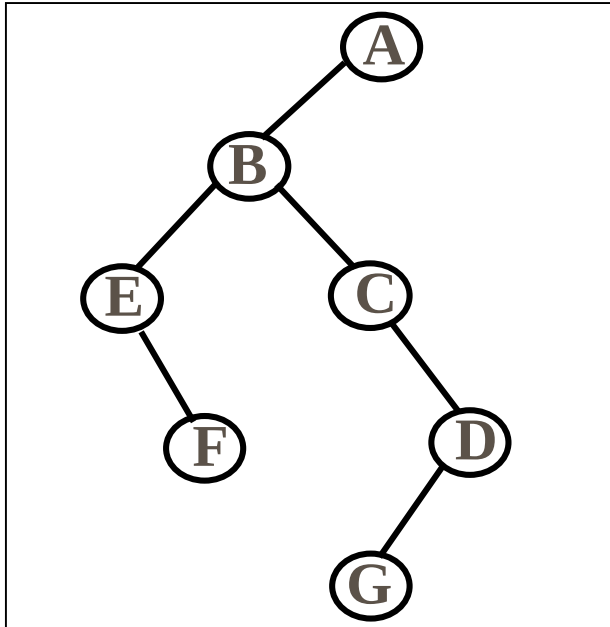
6.4.3 树和森林的遍历



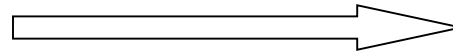
后根遍历



EFBCGDA



中序遍历



EFBCGDA

树的后根遍历等价于二叉树的中序遍历！

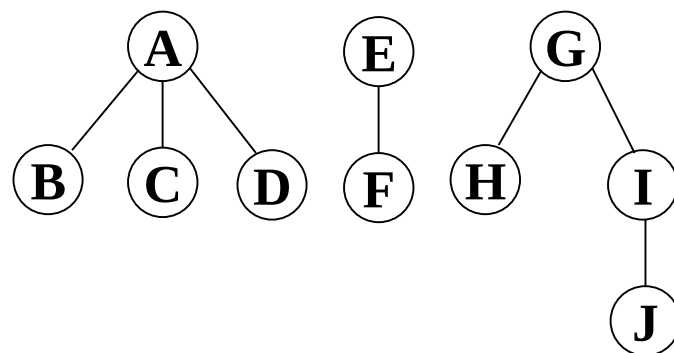
6.4.3 树和森林的遍历

1. 树的先根遍历与二叉树的先序遍历相同；
2. 树的后根遍历相当于二叉树的中序遍历；
3. 树没有中序遍历，因为子树无左右之分。

6.4.3 树和森林的遍历

森林的遍历 { 深度优先遍历 (先序、中序)
 广度优先遍历 (层次)

- **先序遍历**

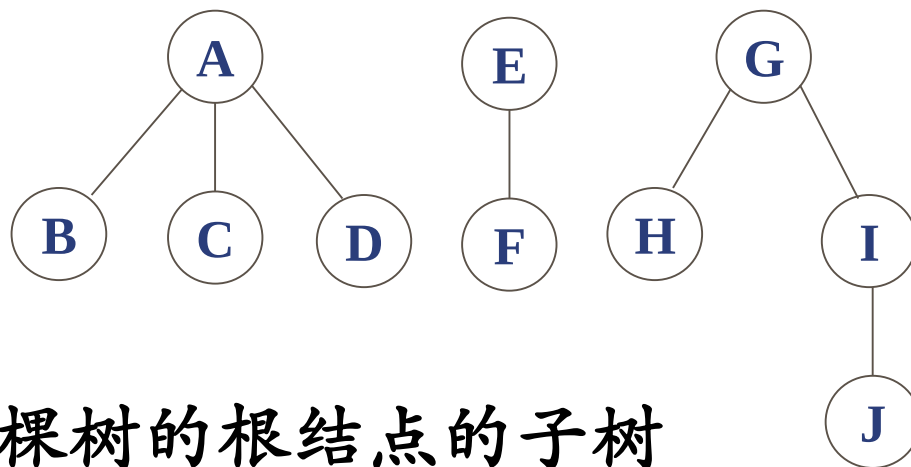


- 若森林为空，返回；
- 访问森林中第一棵树的根结点；
- 先序遍历第一棵树的根结点的子树森林；
- 先序遍历除去第一棵树之后剩余的树构成的森林。

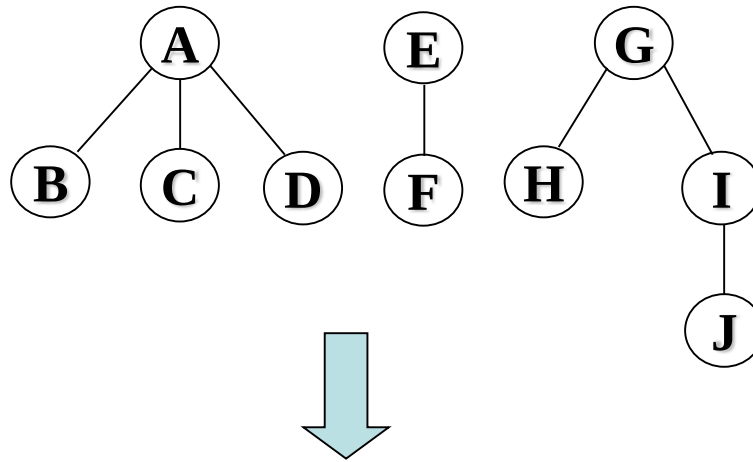
6.4.3 树和森林的遍历

• 中序遍历

- 若森林为空，返回；
- 中序遍历森林中第一棵树的根结点的子树森林；
- 访问第一棵树的根结点；
- 中序遍历除去第一棵树之后剩余的树构成的森林。



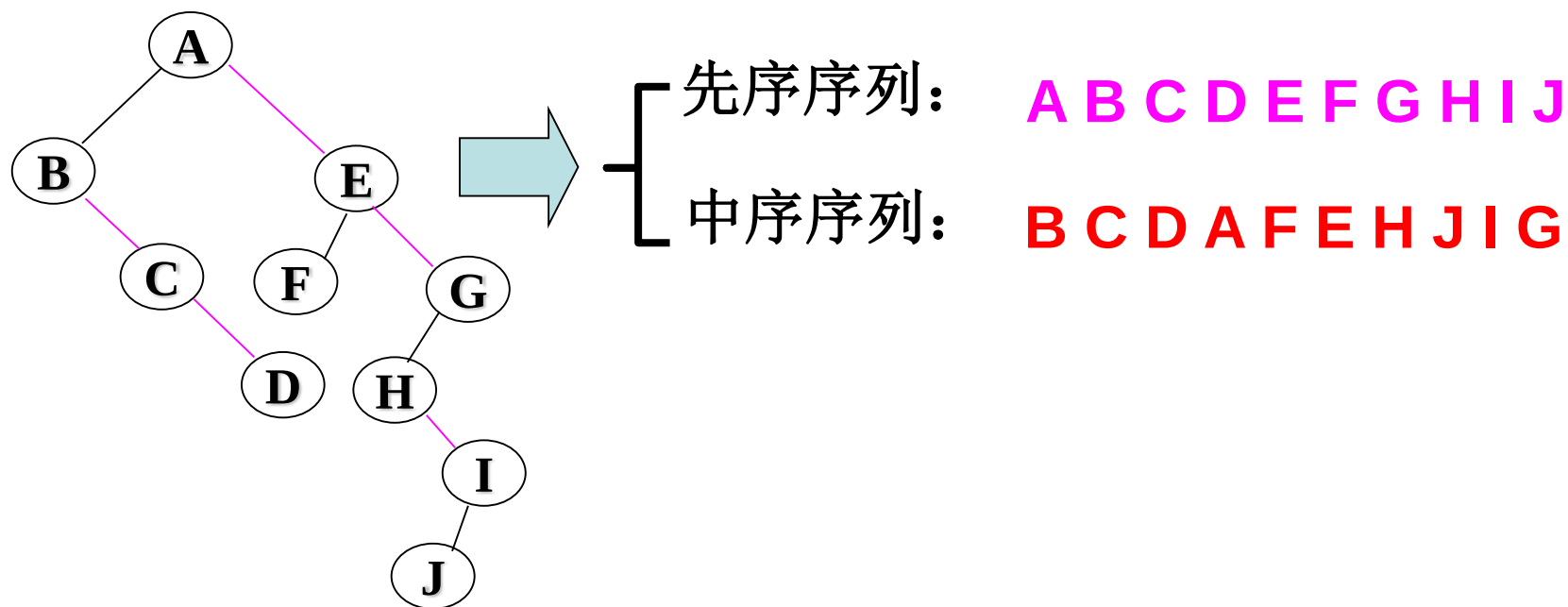
6.4.3 树和森林的遍历



{ 先序序列: **A B C D E F G H I J**
中序序列: **B C D A F E H J I G**

6.4.3 树和森林的遍历

讨论：若采用“先转换，后遍历”方式，结果是否相同？



结论：森林的先序和中序遍历在两种方式下的结果相同。

6.4.3 树和森林的遍历

树的遍历（先根遍历、后根遍历、层次遍历）

（1）先根遍历：先根访问树的根结点，然后依次先根遍历根的每棵子树

（2）后根遍历：先依次后根遍历每棵子树，然后访问根结点

树的先根遍历 - 转换后的二叉树的先序遍历

树的后根遍历 - 转换后的二叉树的中序遍历

森林的遍历

先序：对应二叉树的先序遍历

中序：对应二叉树的中序遍历

6.4.3 树和森林的遍历

设森林F中有三棵树,第一,第二,第三棵树的结点个数分别为 M_1 , M_2 和 M_3 。与森林F对应的二叉树根结点的右子树上的结点个数是()。

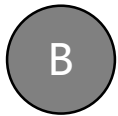
- ☐ A M_1
- ☐ B $M_1 + M_2$
- ☐ C M_3
- ☒ D $M_2 + M_3$

6.4.3 树和森林的遍历

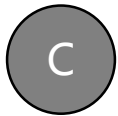
设森林F对应的二叉树为B,它有m个结点,B的根为p, p的右子树结点个数为n, 森林F中第一棵树的结点个数是()



$m-n$



$m-n-1$



$n+1$



条件不足,无法确定

6.4 树和森林

平衡树——特点：所有结点左右子树深度差 ≤ 1

排序树——特点：所有结点“左小右大”

字典树——由字符串构成的二叉排序树

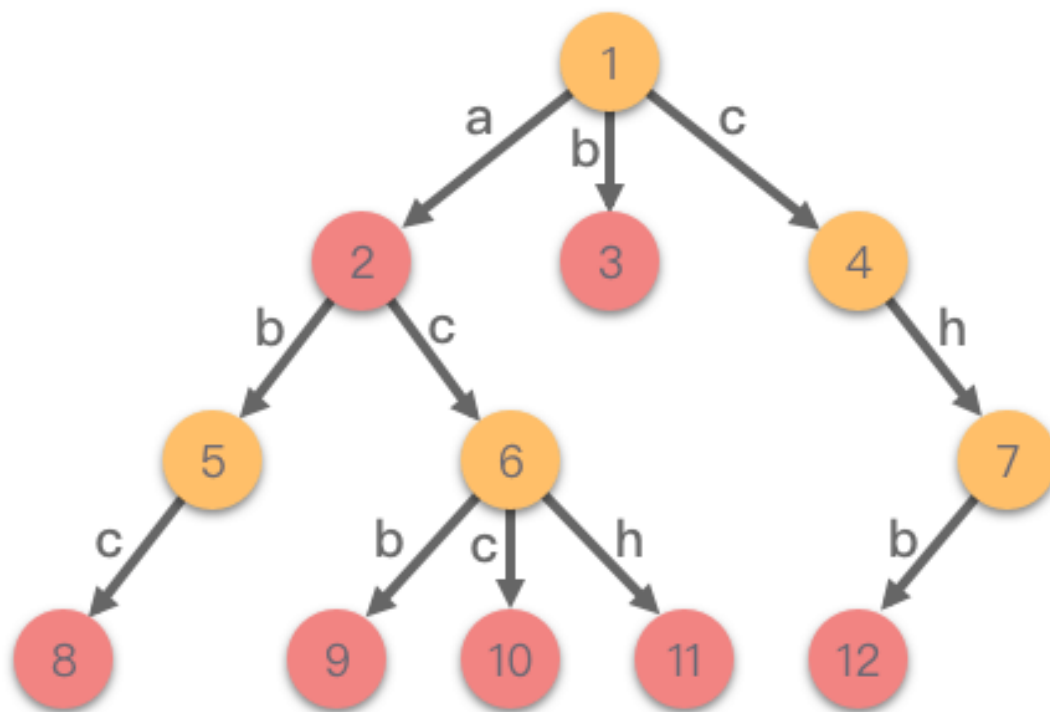
判定树——特点：分支查找树（例如12个球如何只称3次便分出轻重）

带权树——特点：路径带权值（例如长度）

最优树——是带权路径长度最短的树，又称 Huffman 树，用途之一是通信中的压缩编码。

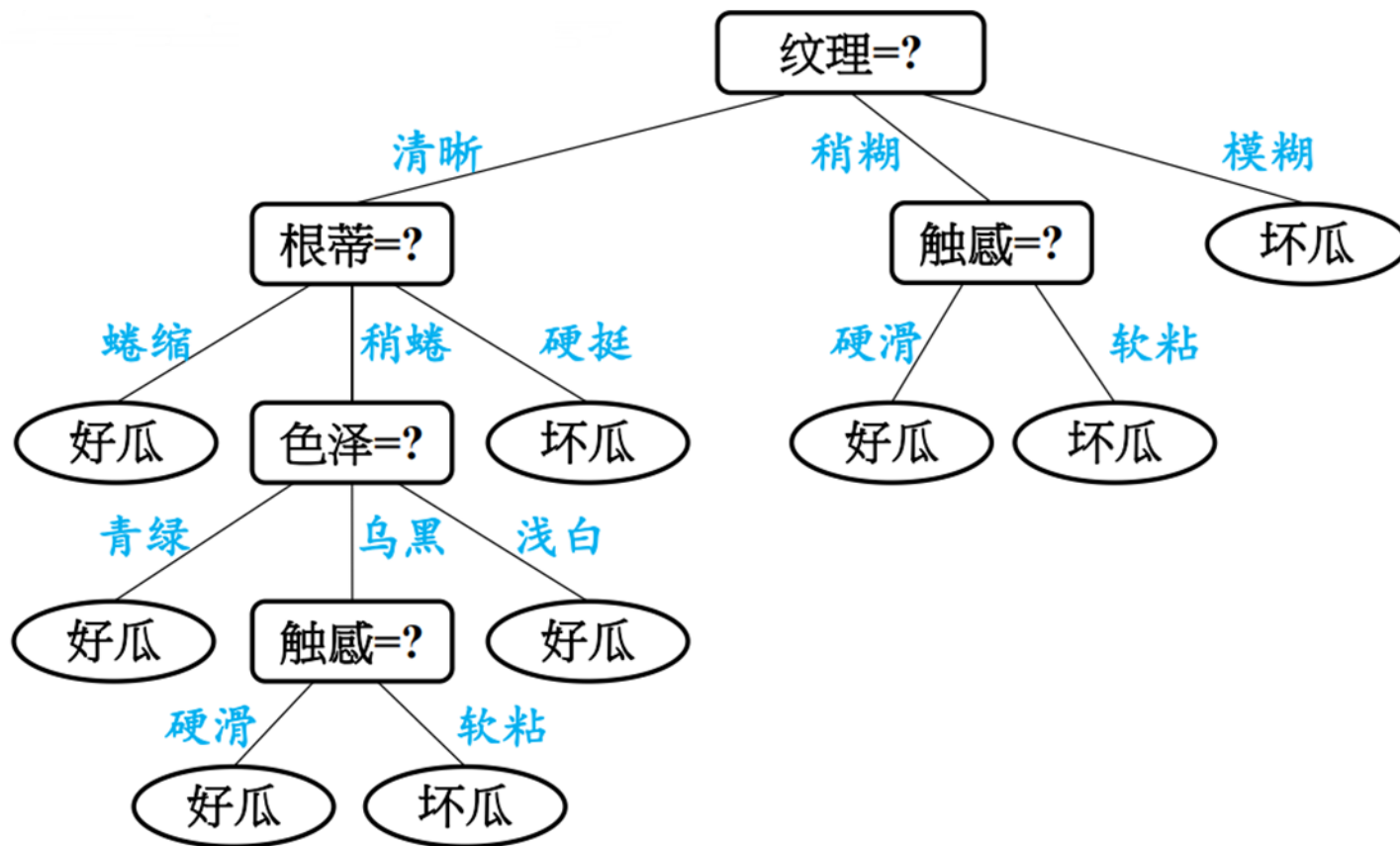
6.4 树和森林

例：下图是一个字典树

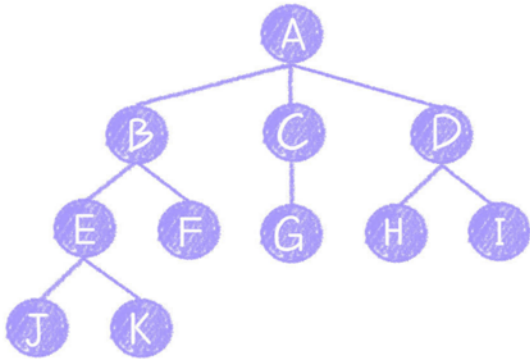
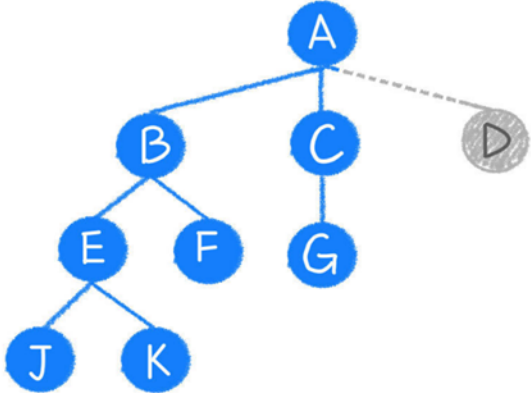


6.4 树和森林

例：下图是一个决策树



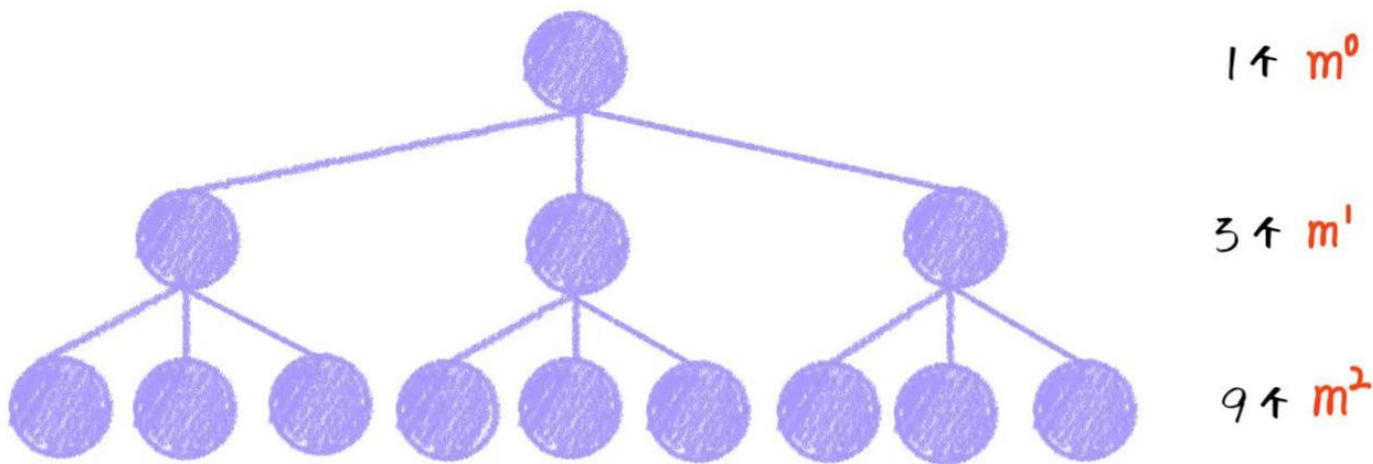
6.4 树和森林

度为m的树 (以度为3的树为例)	m叉树 (以3叉树为例)
	
结点数 = 总度数 + 1 (总度数: 第2层到最后一层的结点; 加1: 根结点)	
各结点的度的最大值	每个结点 最多 只能有m个孩子
任意结点的度 $\leq m$	
至少 有一个结点度 = m	允许 所有 结点的度都 $< m$
一定是非空树, 至少 有 $m + 1$ 个结点	可以是空树
第 i 层 至多 有 m^{i-1} 个结点 ($i \geq 1$)	
高度为 h 、度为m的树 至少 有 $h + m - 1$ 个结点	高度为 h 的 m叉树 至少 有 h 个结点
高度为 h 的树 至多 有 $\frac{m^h - 1}{m - 1}$ 个结点	
n个结点的树的 最小 高度为 $\log_m(n(m - 1) + 1)$	

6.4 树和森林

思考一：为什么度为 m 的树、 m 叉树第 i 层至多有 m^{i-1} 个结点？($i \geq 1$)

以3叉树为例，假如为满三叉树，如下图：



第一层：1个结点， m^0 (m 的0次幂)

第二层：3个结点， m^1

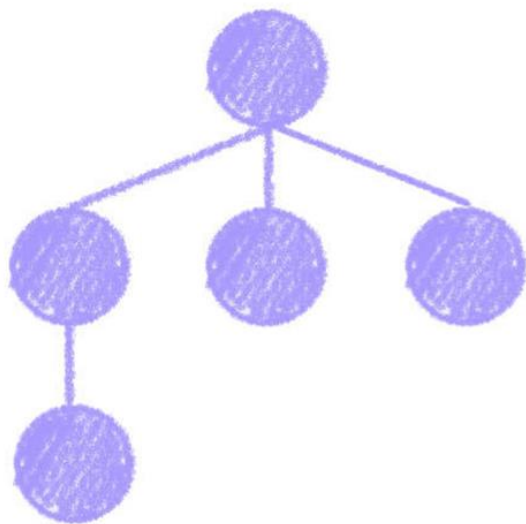
第三层：9个结点， m^2

.....

第 i 层： m^{i-1} 个结点(m 的 $i-1$ 次幂)

6.4 树和森林

思考二：为什么高度为 h ，度为 m 的树至少有 $h+m-1$ 个结点，而 m 叉树至少有 h 个结点？



度为3的树

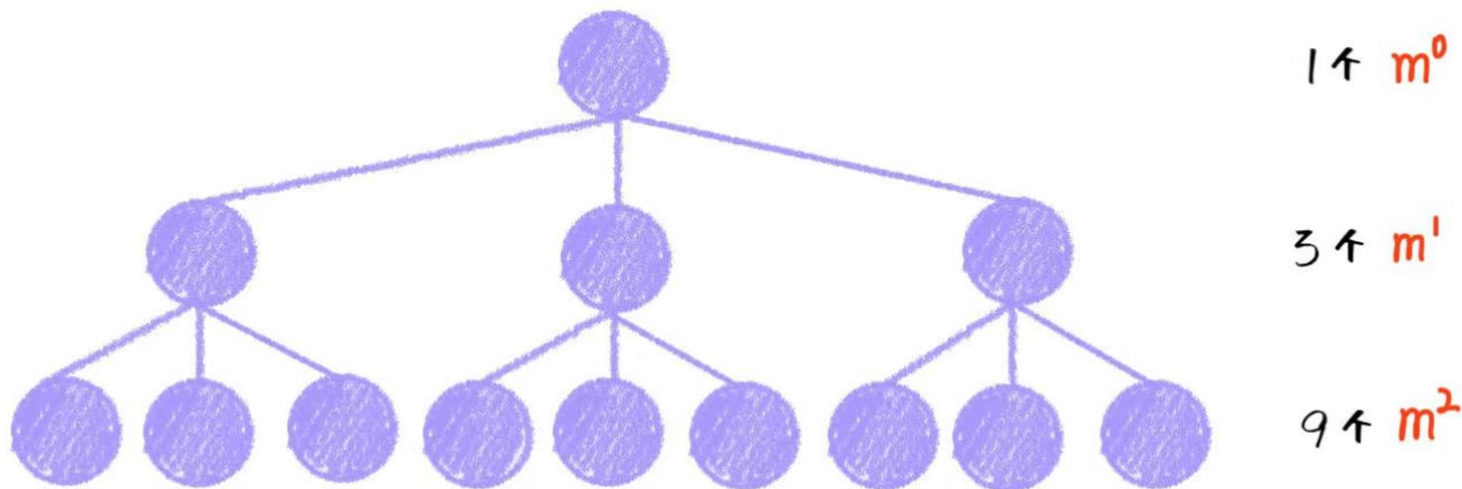


3叉树

1. **度为 m 的树**：各结点的度的最大值——等价于只要有一个结点的度 $= 3$ ，其余的度都最小 $= 1$
2. **m 叉树**：每个结点最多只能有 m 个孩子，允许所有结点的度都 $< m$ ——等价于将每个结点都设成最小 $= 1$

6.4 树和森林

思考三：为什么高度为 h ，度为 m 的树、 m 叉树至多有 $m^h - 1 / m - 1$ 个结点？



高度为 h 为，度为 m 的树、 m 叉树最多有多少个结点：

$$m^0 + m^1 + m^2 + \dots + m^{h-1} = \frac{1*(1-m^h)}{1-m} = \frac{1-m^h}{1-m} = \frac{m^h-1}{m-1}$$

6.4 树和森林

思考四：为什么n个结点的树的最小高度为 $\log_m(n(m-1)+1)$?

前h-1层最多有几个结点

$$\frac{m^{h-1} - 1}{m - 1} < n \leq \frac{m^h - 1}{m - 1}$$

前h层最多有几个结点

$$m^{h-1} < n(m-1) + 1 \leq mh$$

$$h-1 < \log_m(n(m-1)+1) \leq h$$

$$h_{\min} = \lceil \log_m(n(m-1)+1) \rceil$$

例

例：（1）已知一棵度为 m 的树有 n_1 个度为1的结点， n_2 个度为2的结点， $\dots$ ， n_m 个度为 m 的结点，问该树中有多少个叶子结点？

解：设 n 为总结点个数， n_0 为叶子结点（即度为0的结点个数），则有：

$$n = n_0 + n_1 + n_2 + \dots + n_m \quad (1)$$

$$\text{又有（分支总数）： } n - 1 = n_1 * 1 + n_2 * 2 + n_3 * 3 + \dots + n_m * m \quad (2)$$

（因为一个结点对应一个分支）

式(1)-(2)得：

$$1 = n_0 - n_2 - 2n_3 - \dots - (m-1)n_m$$

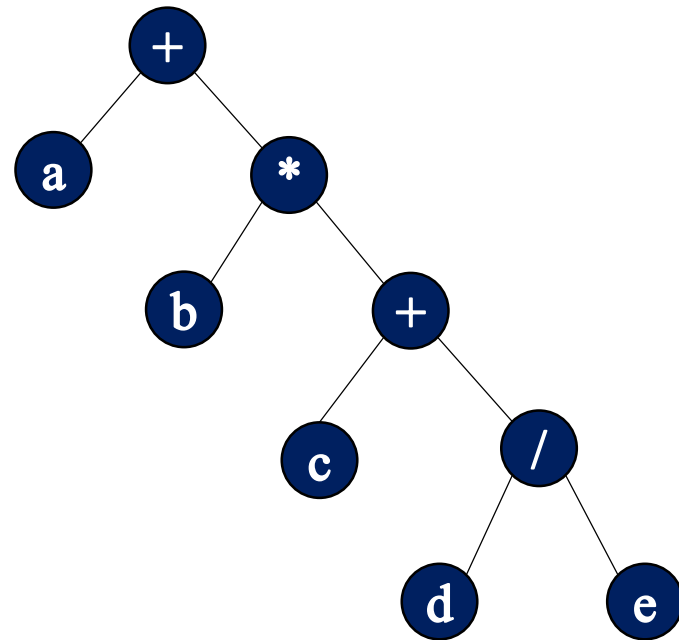
$$\text{则有： } n_0 = 1 + n_2 + 2n_3 + \dots + (m-1)n_m$$

（2）设树 T 的度为4，其中度为1，2，3和4的结点个数分别为4，2，1，1
则 T 中的叶子数为（ 8 ）

例

例：算术表达式 $a+b*(c+d/e)$ 转为后缀表达式后为()

- A $ab+cde/*$
- B $abcde/+*+$
- C $abcde/*++$
- D $abcde^*/++$



例

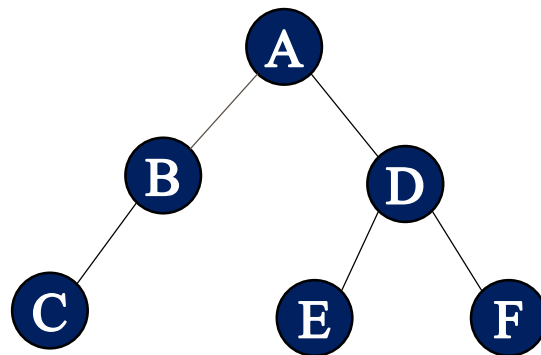
例：已知一棵二叉树的先序遍历结果为ABCDEF,中序遍历结果为CBAEDF,则后序遍历的结果为()。

A CBEFDA

B FEDCBA

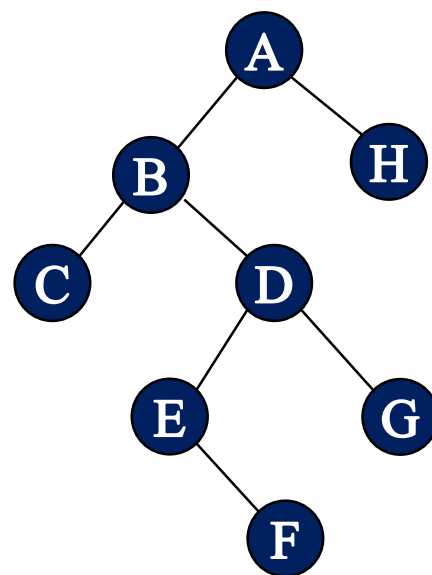
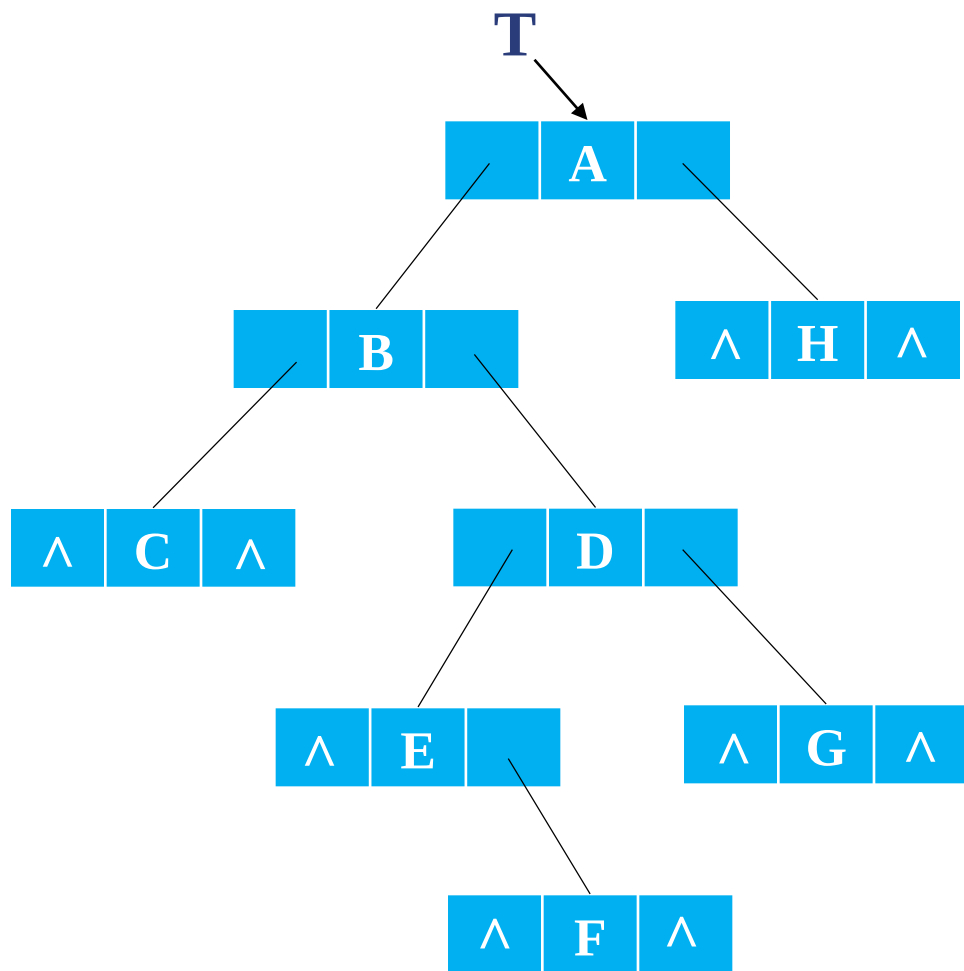
C CBEDFA

D 不定



例

例：输入下面字符串：ABC##DE#F##G##H##，先序输出二叉树



例

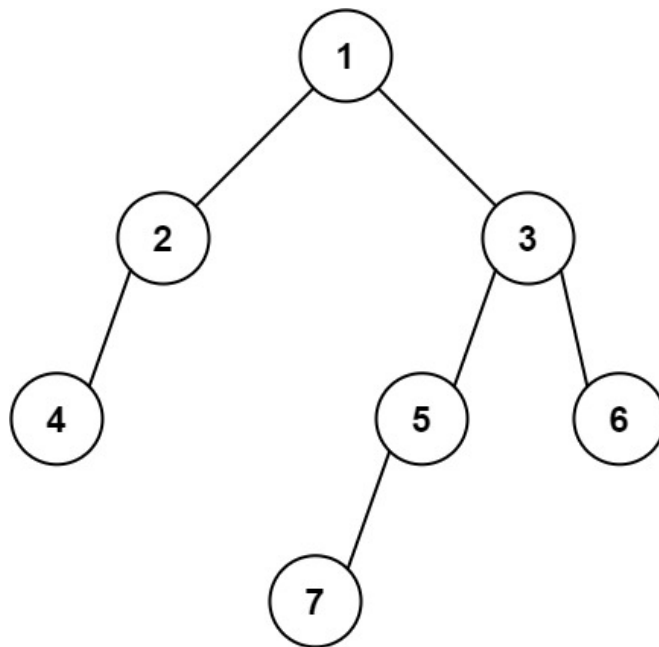
例：给你两棵二叉树的根节点 **p** 和 **q**，编写一个函数来检验这两棵树是否相同。（如果两个树在结构上相同，并且节点具有相同的值，则认为它们是相同的。）

```
bool isSameTree(TreeNode p, TreeNode q) {  
    if(p == null && q == null) return true;  
    if(p == null || q == null) return false;  
    if(p.val != q.val) return false;  
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);  
}
```

例

例：给定一个二叉树的根节点 **root**，请找出该二叉树的最底层最左边节点的值。（假设二叉树中至少有一个节点）

```
void dfs(TreeNode *root, int height, int &curVal, int &curHeight) {  
    if (root == null) return;  
    height++;  
    dfs(root->left, height, curVal, curHeight);  
    dfs(root->right, height, curVal, curHeight);  
    if (height > curHeight) {  
        curHeight = height;  
        curVal = root->val;  
    }  
}  
  
int findBottomLeftValue(TreeNode* root) {  
    int curVal, curHeight = 0;  
    dfs(root, 0, curVal, curHeight);  
    return curVal;  
}
```



例

例：若一棵二叉树，左右子树均有三个结点，其左子树的先序序列与中序序列相同，右子树的中序序列与后序序列相同，试构造该树。

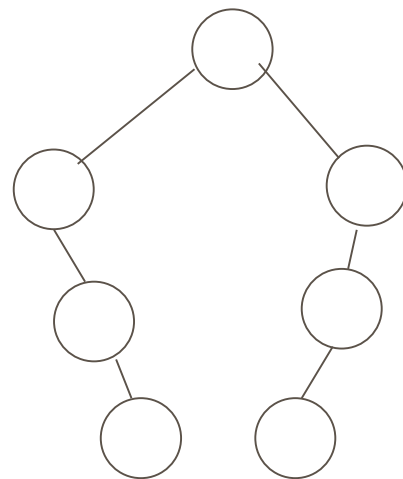
【解】据题意，左子树的先序序列与中序序列相同，即有：

先序：	根	左	右
中序：	左	根	右

也即，以左子树为根(tree)无左孩子。此处，右子树的中序序列与后序序列相同，即有：

中序：	左	根	右
后序：	左	右	根

也即，以右子树为根(tree)无右孩子。
由此构造该树如下图所示。

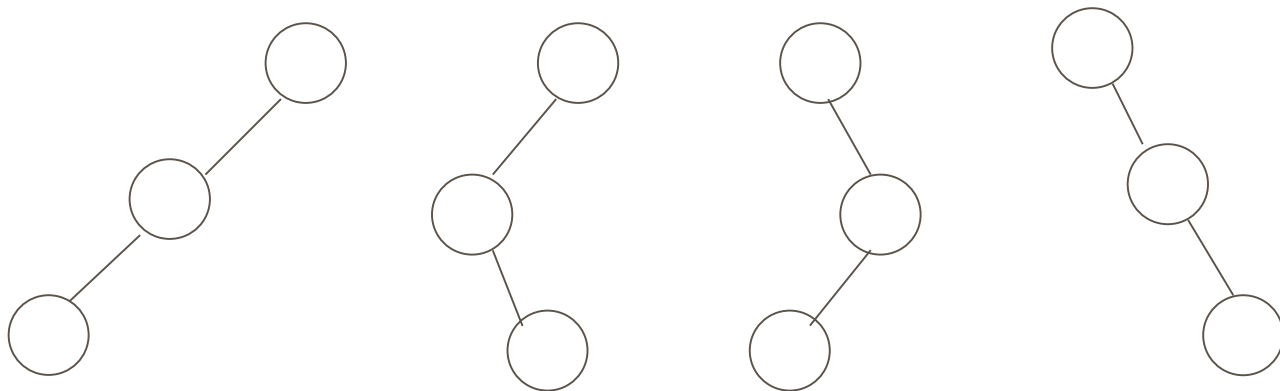


例

例：一棵非空的二叉树其先序序列和后序序列正好相反，画出这棵二叉树的形状

解：先序遍历为“根左右”，后序遍历为“左右根”。

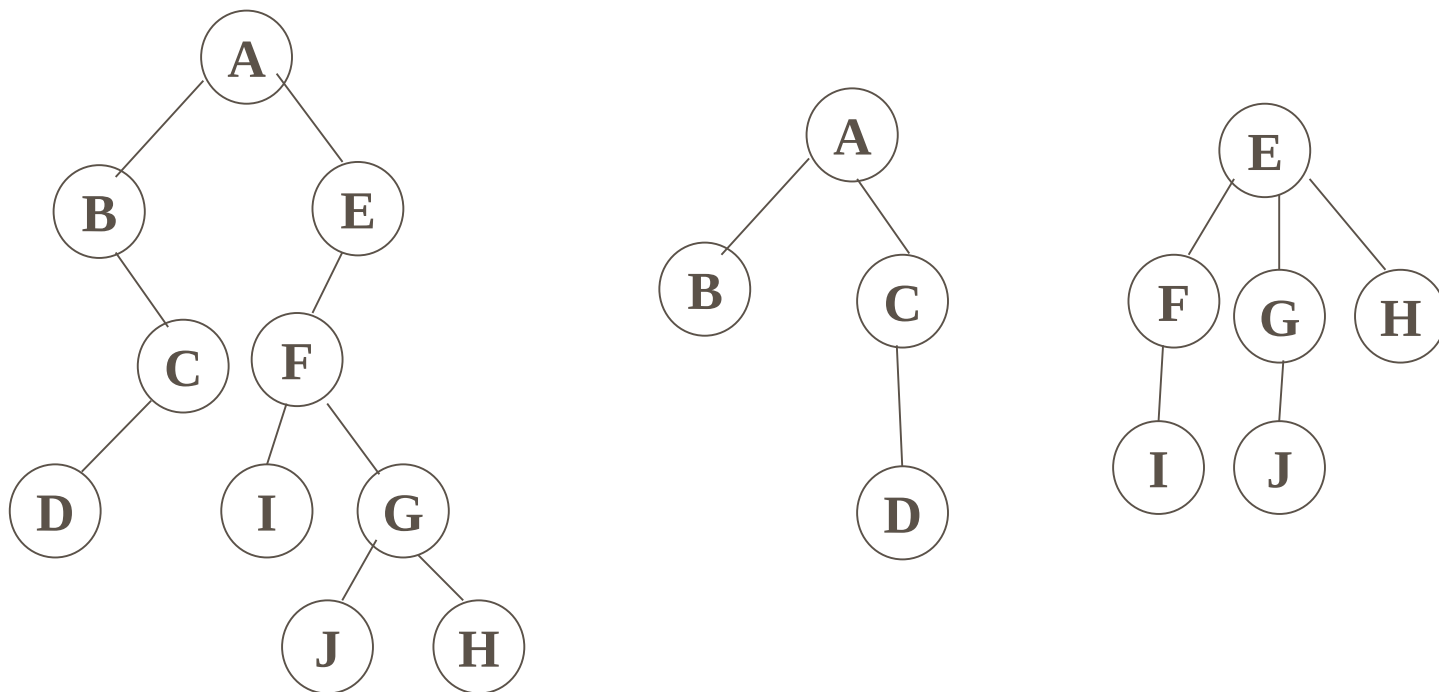
根结点在两个序列中的位置分别在最前和最后，正好相反；因此，若要两个序列正好相反，则左右子树必有一个不存在。其先序序列和后序序列正好相反的二叉树必为单支树。即这棵二叉树的形状如下图所示。



例

例:如果已知森林的前序序列和后序序列分别为ABCDEFIJGH和BDCAIFJGHE, 请画出该森林。

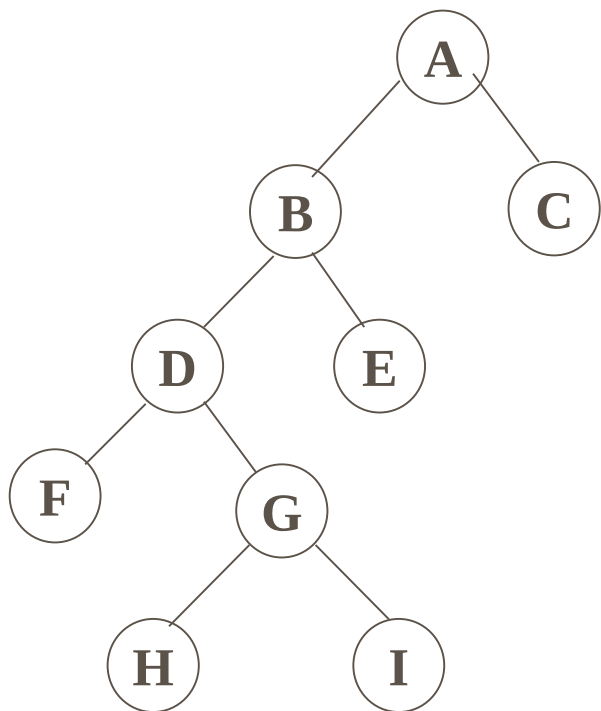
【解】由于森林的前序序列与其对应的二叉树前序序列相同, 而森林的后序序列与其对应的二叉树中序序列相同。因此, 根据二叉树前序序列ABCDEFIJGH和中序序列BDCAIFJGHE可画出二叉树如下图所示。



例

例:对如下图所示的二叉树:

- (1) 写出对它们进行先序、中序和后序遍历时得到的结点序列;
- (2) 画出它们的先序线索二叉树和后序线索二叉树。



【解】

先序遍历的结点序列为: ABDFGHIEC

中序遍历的结点序列为: FDHGIBEAC

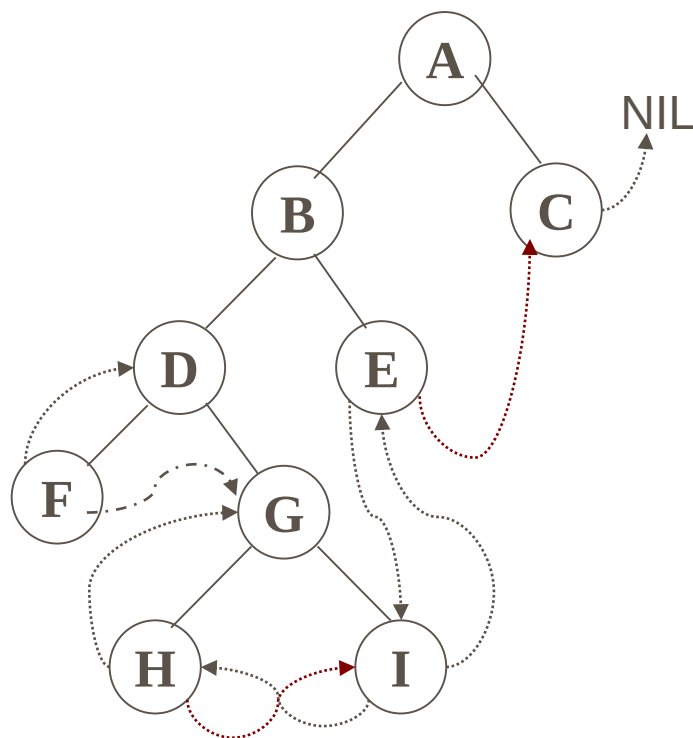
后序遍历的结点序列为: FHIGDEBCA

例

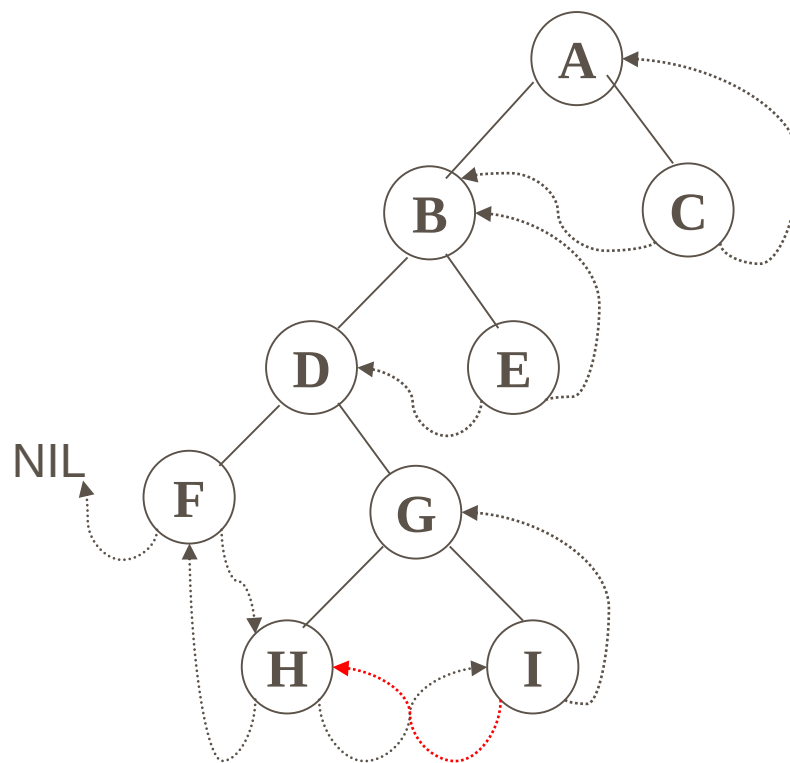
二叉树的先序线索二叉树如下左图所示，后序线索二叉树如下右图所示。

先序遍历：ABDFGHIEC

后序遍历：FHIGDEBCA



先序线索二叉树



后序线索二叉树

正在答疑
